Click cram sheet: building CLIs

pallets/click@2247b35 (8.6.0.dev) · 2026-11-02 · Commands checked against source, not executed

If you only read one thing: a decorated function is now a `Command`. Calling `hello("x")` parses `"x"` as argv and exits; keep logic in a plain function and call that.

Legend: ★ most used · ⚠ gotcha

START

```
pip install click
```

```
import click

@click.command()
@click.option("-n",
              default=1)
@click.argument("name")
def hello(n, name):
    """Greet NAME."""
    for _ in range(n):
        click.echo(name)
```

- Run as a script: `if __name__ == "__main__": hello()`
- The docstring becomes `--help` text.

OPTIONS ★

- `default=1` → also sets type int
- `required=True` → error if missing
- `is_flag=True` → `True` when passed
- `"--x/--no-x"` → on/off pair
- `multiple=True` → repeatable, tuple
- `count=True` → `-vvv` gives 3
- `envvar="TOKEN"` → read env var
- `prompt=True` → ask if missing
- `hide_input=True` → for passwords
- `show_default=True` → in help

NAMES

- `"--dry-run"` → param `dry_run`
- `"-n", "--name", "who"` → param `who`
- function `sync_cmd` → command `sync`; suffixes `_command`, `_cmd`, `_group`, `_grp` drop

TYPES

- `click.Choice(["a", "b"])` → one of list
- `click.IntRange(0, 9)` → bounded int
- `click.Path(exists=True)` → checked path str
- `click.File("r")` → open file; `-` is stdin
- `type=(str, int)` → tuple, nargs=2

ARGUMENTS

- Required by default.
- `help=` needs 8.5+; before that, describe it in the docstring.
- `nargs=-1` → tuple of any length; once per command.
- `cmd -- -odd.txt`: after `--`, all are arguments.

GROUPS AND CONTEXT

```
@click.group()
@click.option("--debug",
              is_flag=True)
@click.pass_context
def cli(ctx, debug):
    ctx.ensure_object(dict)
    ctx.obj["dbg"] = debug

@cli.command()
@click.pass_obj
def sync(obj):
    click.echo(obj["dbg"])
```

- `cli.add_command(cmd)` registers a command from another module.
- `invoke_without_command=True` runs the group with no subcommand; check `ctx.invoked_subcommand`.

SHIP IT

```
[project.scripts]
hello = "pkg.cli:hello"
```

- `pip install -e .`, then run `hello`.
- Zsh completion: `_HELLO_COMPLETE=zsh_source hello` prints a script; `eval` it in `.zshrc`.

ERRORS AND EXIT CODES

- `raise click.ClickException(msg)` → exit 1
- `click.UsageError`, `click.BadParameter` → exit 2
- `click.Abort` → prints `Aborted!`, exit 1
- `click.confirm("Go?", abort=True)` raises `Abort` on no.
- `--help` exits 0.
- `hello(["Ann"], standalone_mode=False)` returns the value and lets errors raise instead of exiting.

TEST

```
from click.testing import (
    CliRunner)

r = CliRunner().invoke(
    hello, ["Ann"])
assert r.exit_code == 0
assert r.output == "Ann\n"
```

`r.output` mixes stdout and stderr (8.2+); use `r.stdout` for one stream. Exceptions are caught: check `r.exception`.

GOTCHAS

- ⚠ `-dbg` means `-d -b -g`. Short names are one letter, so use `--dbg`.
- ⚠ `multiple=True, default="ab"` gives `("a","b")`, because a string is a sequence. Use a list.
- ⚠ `--x/--no-x` with `envvar="X"` reads only `X`; there is no `NO_X`.
- ⚠ `auto_envvar_prefix="APP"` names include the subcommand: `APP_RUN_HOST`, not `APP_HOST`.
- ⚠ A group run with no args shows help and exits 2, not 0.
- ⚠ `print()` can raise `UnicodeError` on a broken terminal; `click.echo` corrects it.
- ⚠ `CliRunner` swaps global stdio: it is not thread-safe.

OUTPUT

- `click.echo(msg, err=True)` → stderr
- `click.secho("ok", fg="green")` → styled; plain when piped
- `click.prompt("Port", type=int)` asks and converts
- A line with only `\b` stops help rewrapping the next paragraph.
- `@click.version_option()` adds `--version` from package metadata.
- `context_settings={"help_option_names": ["-h", "--help"]}` adds `-h`.